

Embedded Software Engineering

Simon Künzli, Tobias Welti
HS 2025

Agenda HS25

Vorlesung (welo)

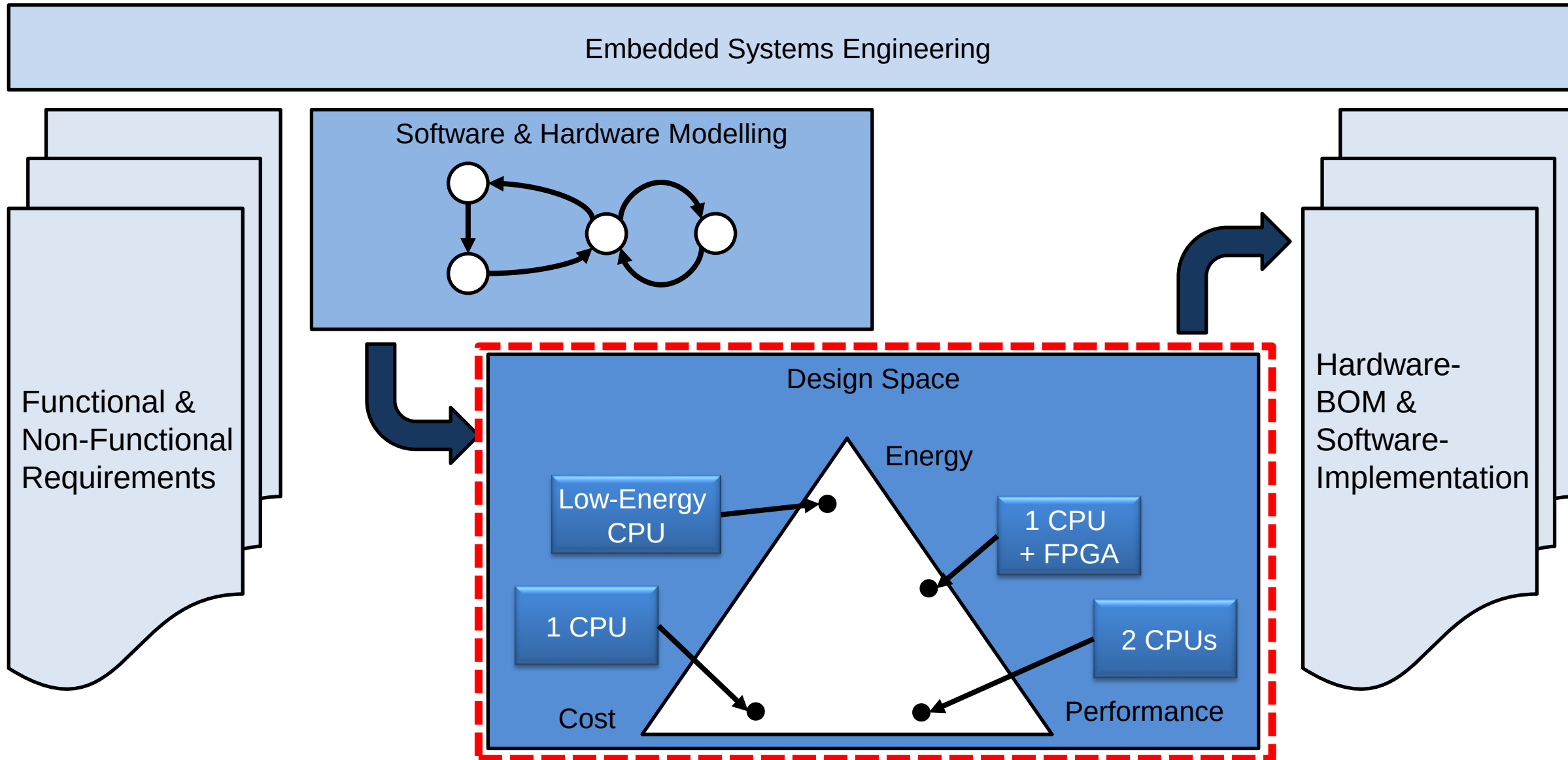
Vorlesung (kuex)

Übungen / Praktika

Stand:
27.08.25

wk #	Do, 10:00 - 11:35	Do, 08:00 – 09:35 / Do 12:00 – 13:35	
01 15.09.	Modulorganisation & Einführung Embedded Systems	No Lab	
02 22.09.	SW-Paradigmen für Embedded Systems	No Lab	P1: First steps U96
03 29.09.	Requirements für Embedded Systems	P1: First steps U96	P1: Finish Lab
04 06.10.	Software-Entwicklung für ES (Modellierung, SW-Specs)	P1: Finish Lab	U1: Requirements
05 13.10.	Entwicklungsprozesse / CI/CD	U1: Requirements	U2: Modeling
06 20.10.	Non-functional Requirements	U2: Modeling	U3: Implementation & Test
07 27.10.	Visite Fertigung Siemens	Visite Fertigung Siemens	
08 03.11.	Fokus "Energie« / andere Prozessoren?	U3: Implementation & Test	P2: Energy Consumption
09 10.11.	Fokus "Performance"	P2: Energy Consumption	P3: AES in SW, incl. Measurement on R5
10 17.11.	Einführung FPGA	P3: AES in SW, incl. Measurement on R5	P3: Finish Lab
11 24.11.	Einführung Design Space Exploration (DSE)	P3: Finish Lab	P4: AES in FPGA and HW-Accelerator
12 01.12.	DSE: Kostenfunktionen und Suche im Entwurfsraum	P4: AES in FPGA and HW-Accelerator	P4: Finish Lab
13 08.12.	Einführung RTOS	P4: Finish Lab	P5: DSE
14 15.12.	Dual-Prozessor-Systeme mit Uniform Memory Access	P5: DSE	No Lab

Advance Organizer ESE

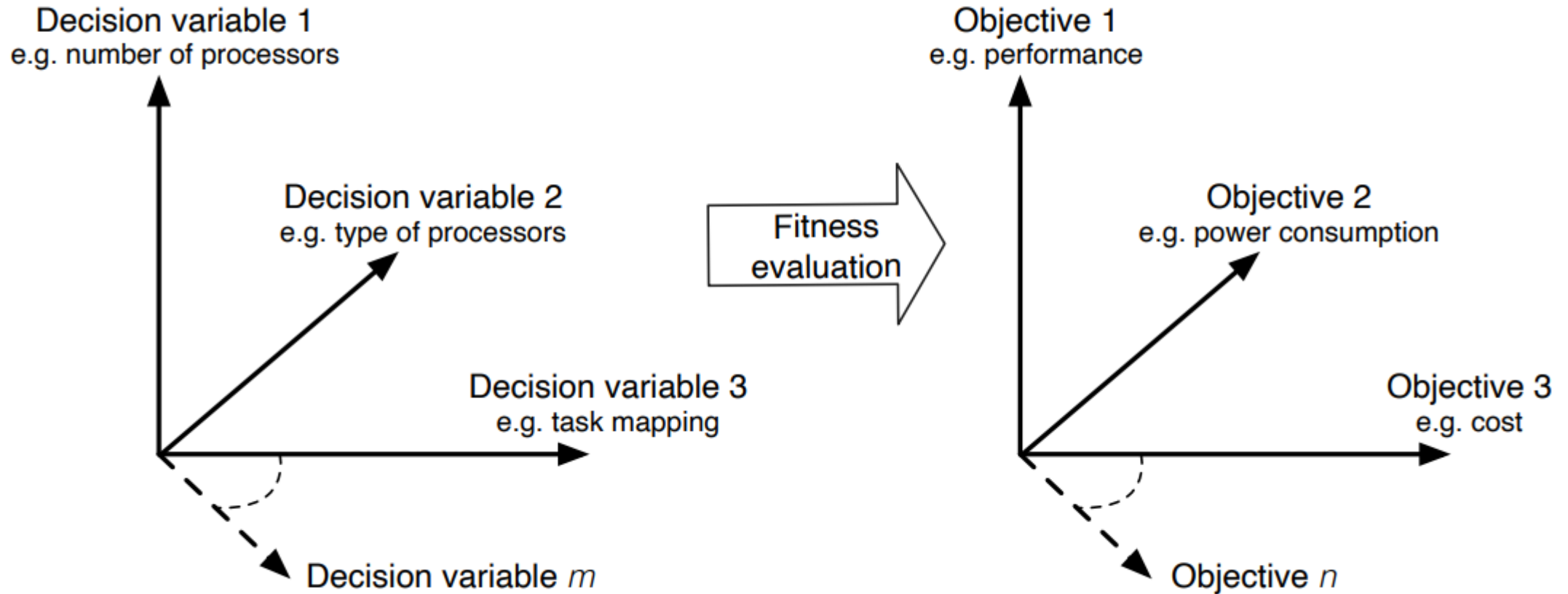


Learning Objectives

After today's lesson, you ...

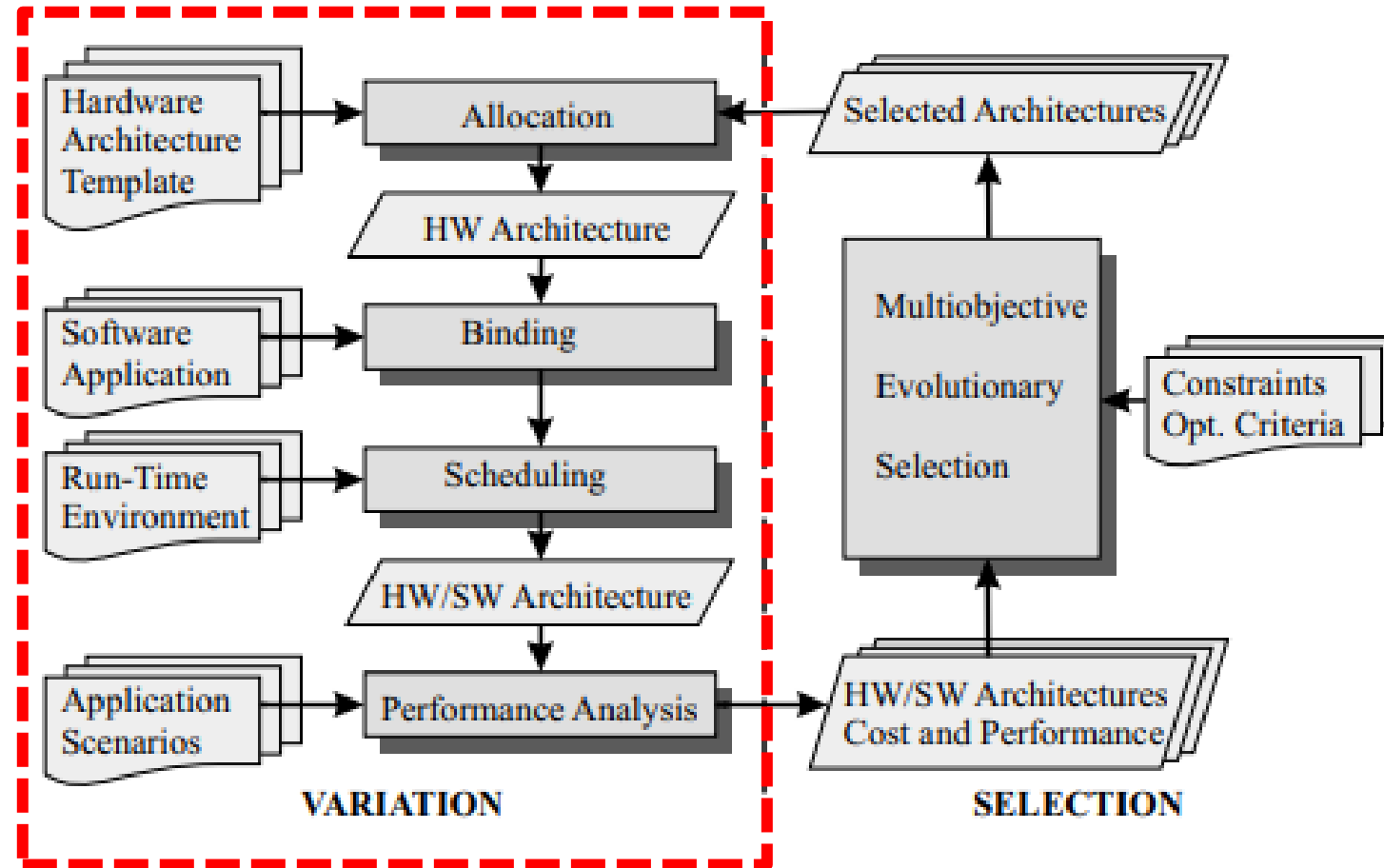
- ... will be able to explain what design space exploration is about
- ... know methods of how different solutions for ES can be found
- ... can explain the basic principles of evolutionary algorithms and their field of application
- ... know how to compare various Embedded System solutions
- ... will be able to perform a design space exploration

Recap: Design Space Exploration



Source: A.D.Pimentel: Methodologies for Design Space Exploration

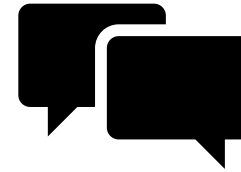
Summary & Outlook



Search

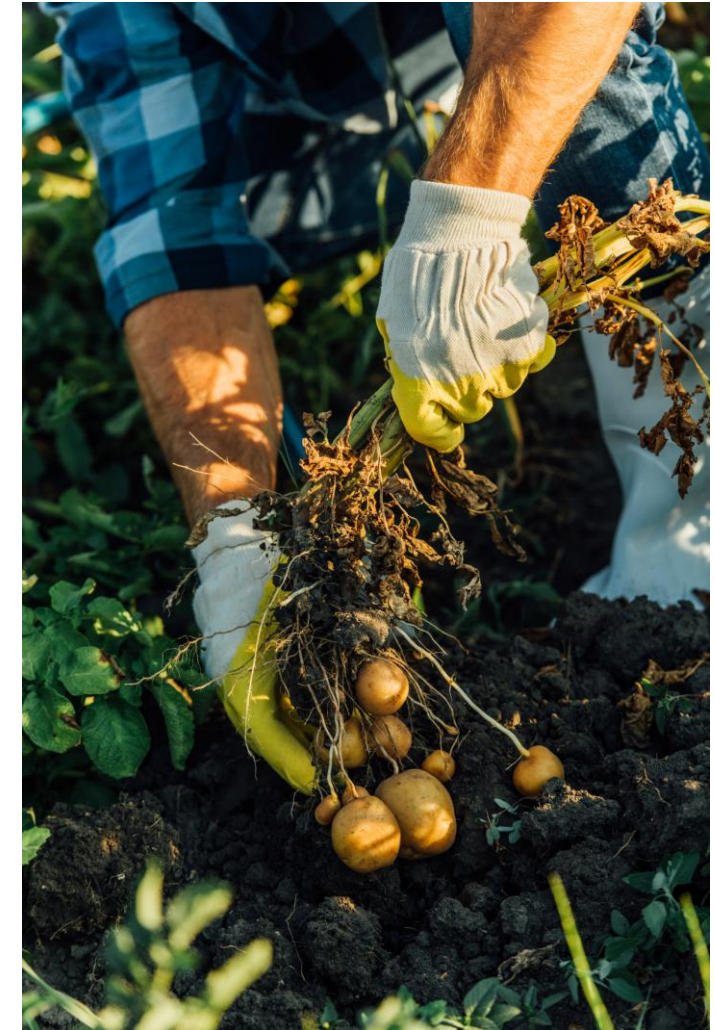
How can we find the best solution(s)?





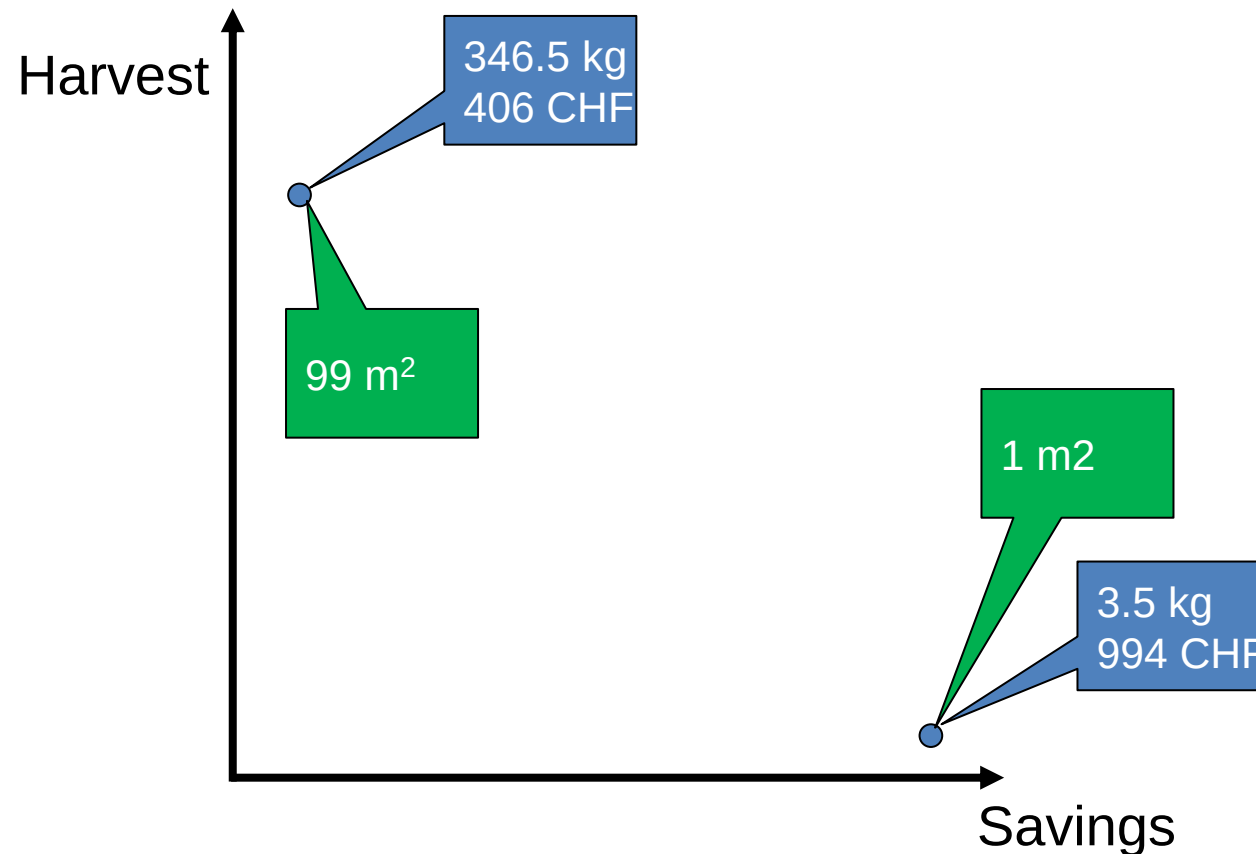
Example 1: Potato Field

- Assume a potato field of 100m^2
- On how many m^2 shall we plant potatoes to optimize our savings and harvest?
- What is our design variable?
- What are our objectives?
- Do we have constraints?



Example 1: Potato Field

- **Linear Dependency between decision space and objective space**



$1\text{m}^2 \leftrightarrow 3.5 \text{ kg}$

Source: <https://www.wurzelwerk.net/gemuesegarten/kartoffeln/selbstversorgung-kartoffeln/>

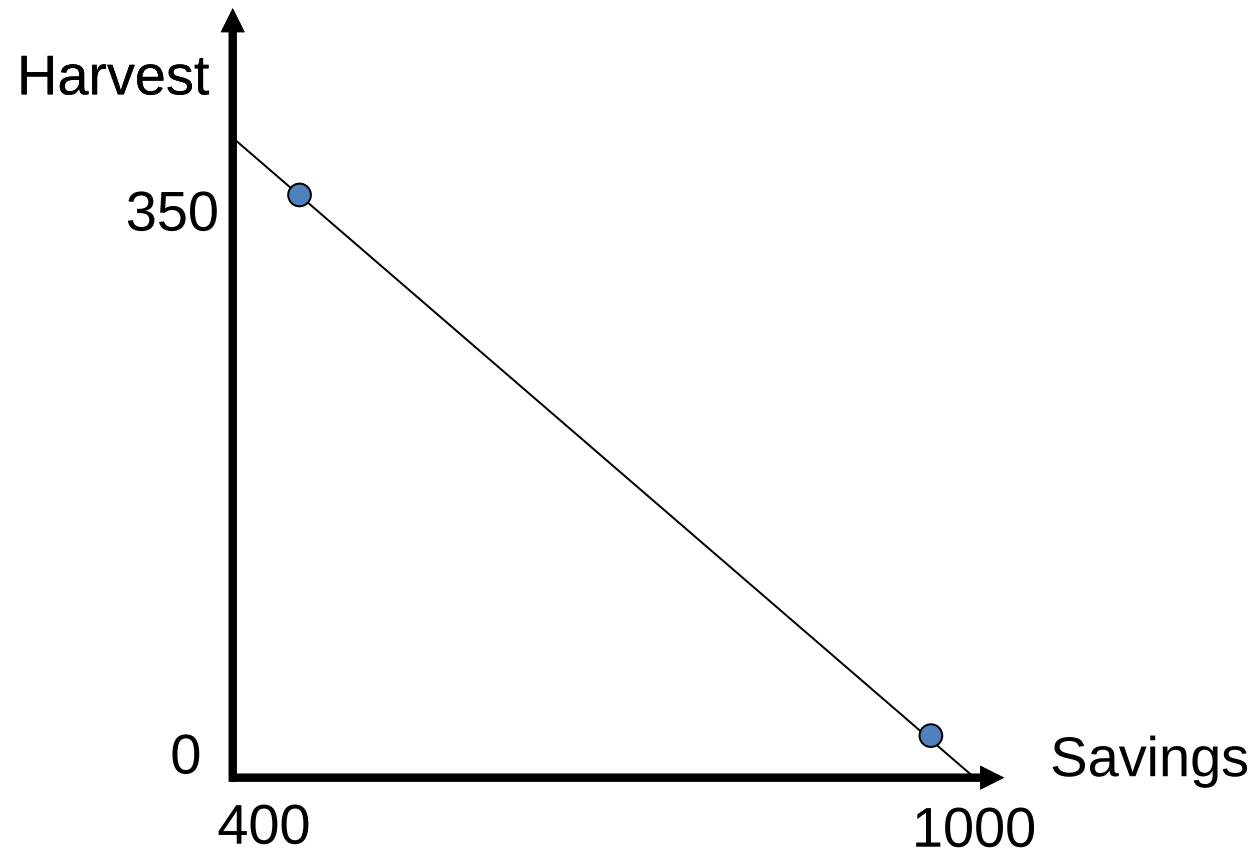
$1\text{m}^2 \leftrightarrow 6 \text{ CHF}$

Source: https://www.zbv.ch/uploads/1/2/0/2/120245015/fachteil__vom_30.08.2019.pdf

Assumption: 1000 CHF available

Example 1: Potato Field

- **Linear Dependency between decision space and objective space**
→ **Simple Search, calculate Pareto-Optimal Front**



$1\text{m}^2 \leftrightarrow 3.5 \text{ kg}$

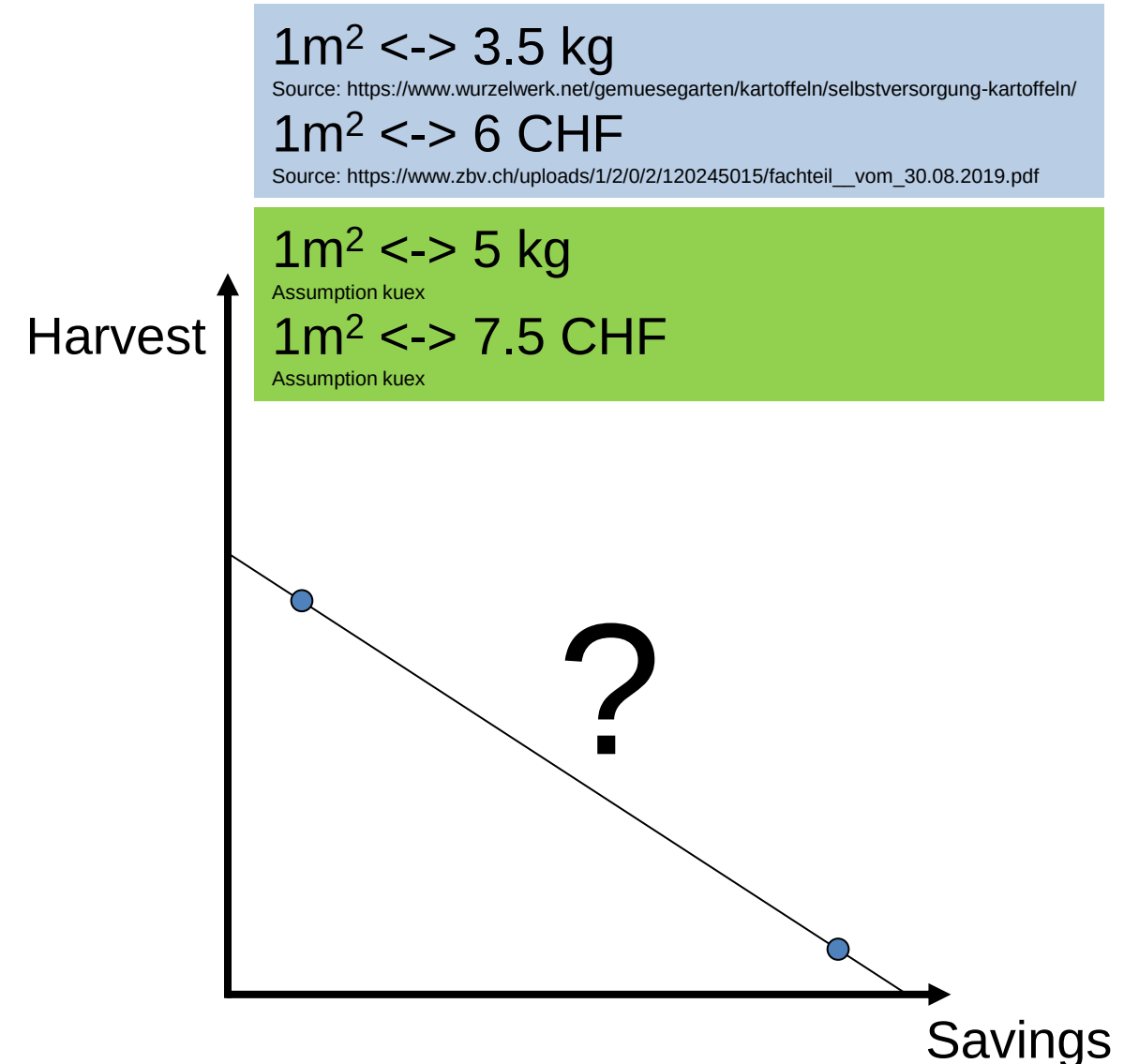
Source: <https://www.wurzelwerk.net/gemuesegarten/kartoffeln/selbstversorgung-kartoffeln/>

$1\text{m}^2 \leftrightarrow 6 \text{ CHF}$

Source: https://www.zbv.ch/uploads/1/2/0/2/120245015/fachteil__vom_30.08.2019.pdf

Example 1: Potato Field

- **What if we add fertilizers to the problem?**
 - Add fertilizers for 1.5 CHF/m²
 - Fertilizers increase Harvest to 5kg/m²
- **How does the Pareto-Optimal Front look like now?**
 - Is there still a linear relationship between decision and objective values?
 - Is it true that the more fertilizers we use, the more possible harvest?
 - What about the savings?



Example 1: Potato Field

■ What if we add fertilizers to the problem?

- Add fertilizers for 1.5 CHF/m²
- Fertilizers increase Harvest to 5kg/m²

■ Problem Formulation:

- Decision variables:
 - x_1 : area without fertilizer
 - x_2 : area with fertilizer
- Constraints:
 - $x_1 + x_2 \leq 100 \text{ m}^2$
- Objectives
 - Harvest: $f_1 = x_1 * 3.5 + x_2 * 5$
 - Savings: $f_2 = 1000 - x_1 * 6 - x_2 * 7.5$

$1\text{m}^2 \leftrightarrow 3.5 \text{ kg}$

Source: <https://www.wurzelwerk.net/gemuesegarten/kartoffeln/selbstversorgung-kartoffeln/>

$1\text{m}^2 \leftrightarrow 6 \text{ CHF}$

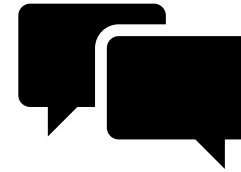
Source: https://www.zbv.ch/uploads/1/2/0/2/120245015/fachteil__vom_30.08.2019.pdf

$1\text{m}^2 \leftrightarrow 5 \text{ kg}$

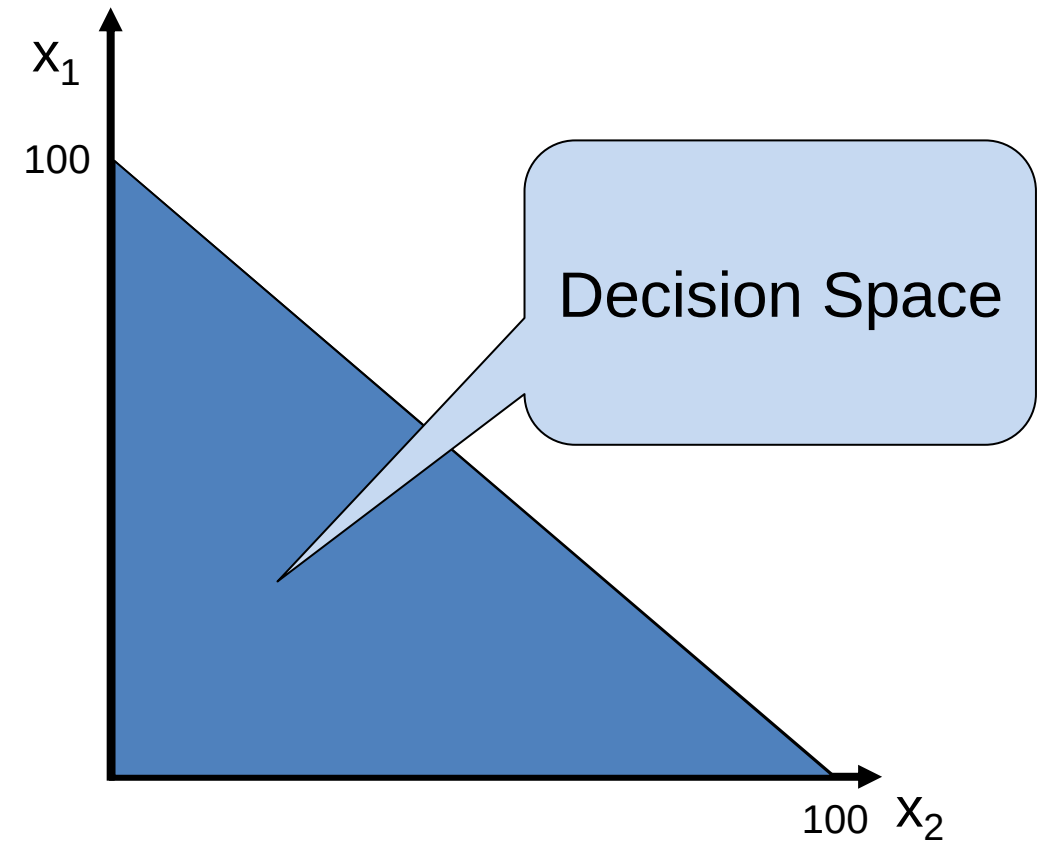
Assumption kuex

$1\text{m}^2 \leftrightarrow 7.5 \text{ CHF}$

Assumption kuex

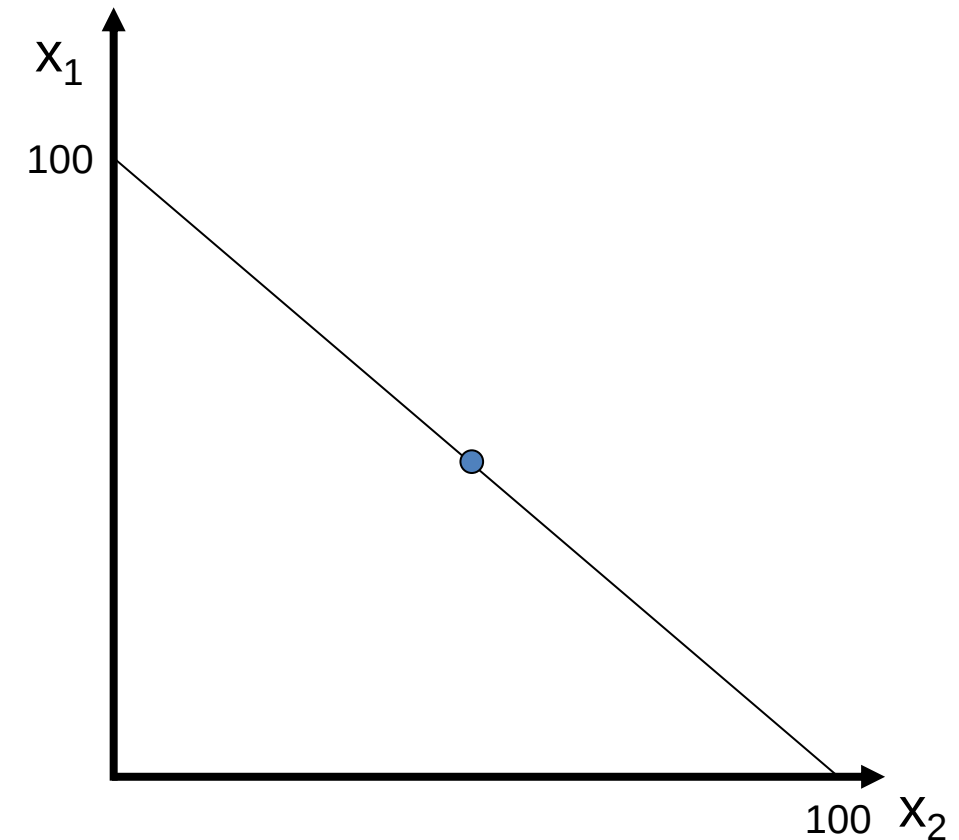


- Possible use of linear optimization solver
- <https://online-optimizer.appspot.com/>



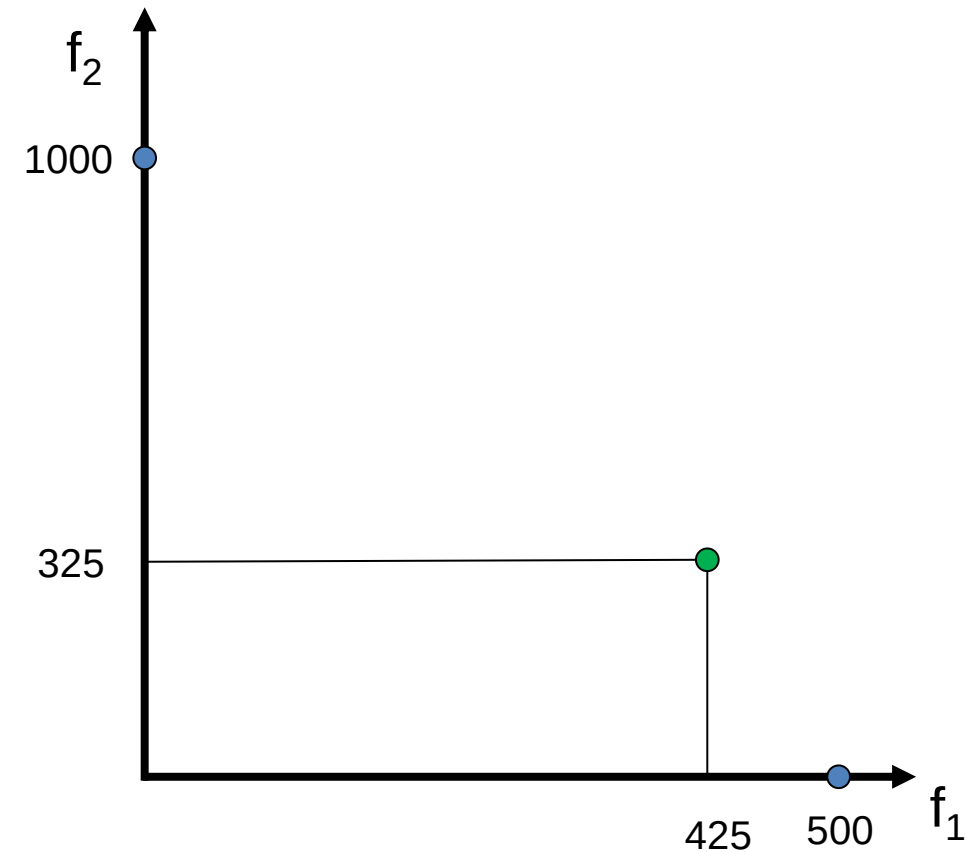
Optimization – Decision Space

- **Possible use of linear optimization solver**
- **Let's have a look at this solutions:**
 - x_1 (area without fertilizer): 50.00 m²
 - x_2 (area with fertilizer): 50.00 m²
 - Harvest value (f_1): 425.00
 - Savings value (f_2): 325.00
- **This is a pareto-optimal solution...**
 - ... if you want to optimize the **sum** of f_1 and f_2
 - ... and **all** the land must be used for potatoes

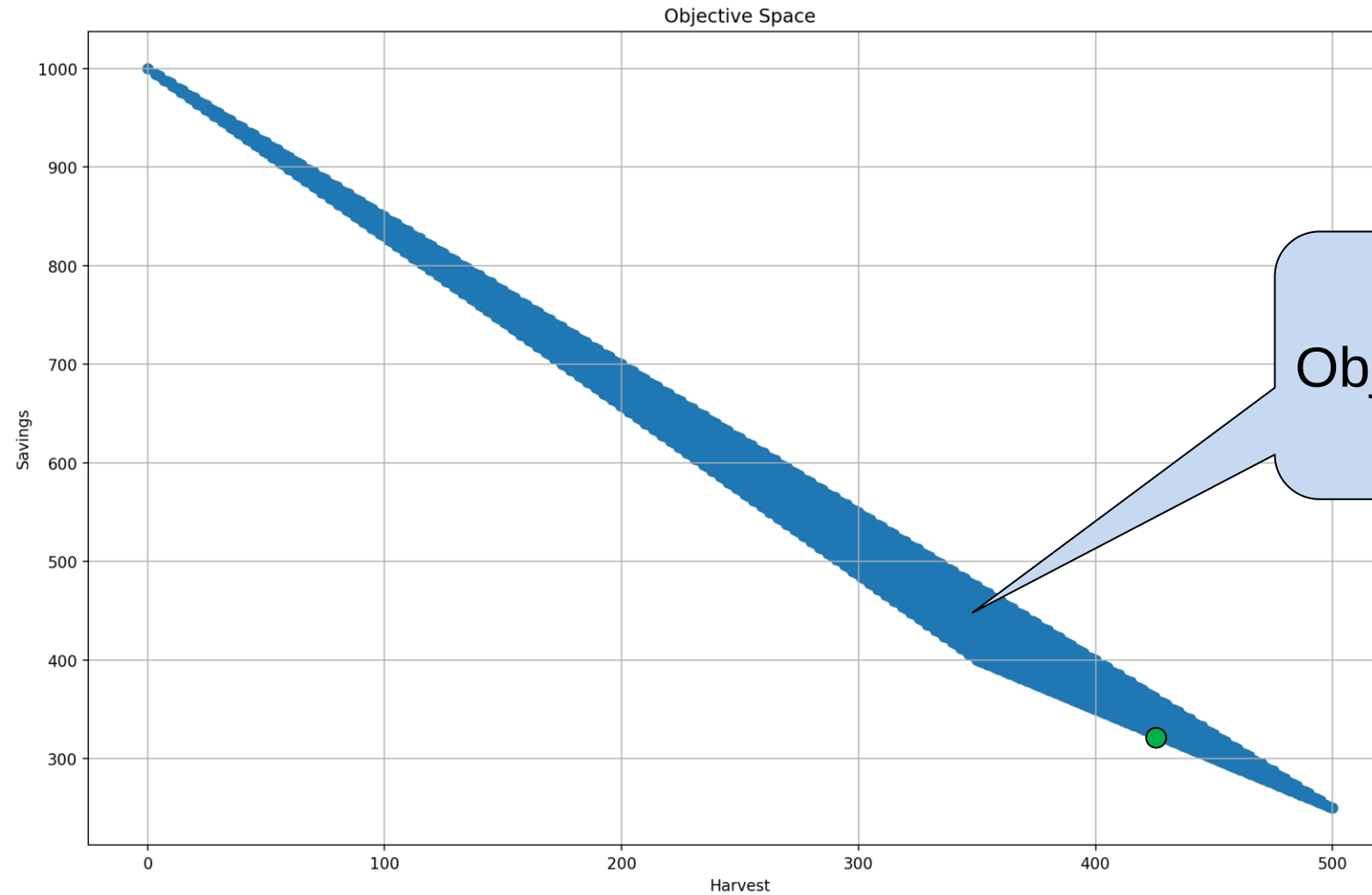


Optimization – Objective Space

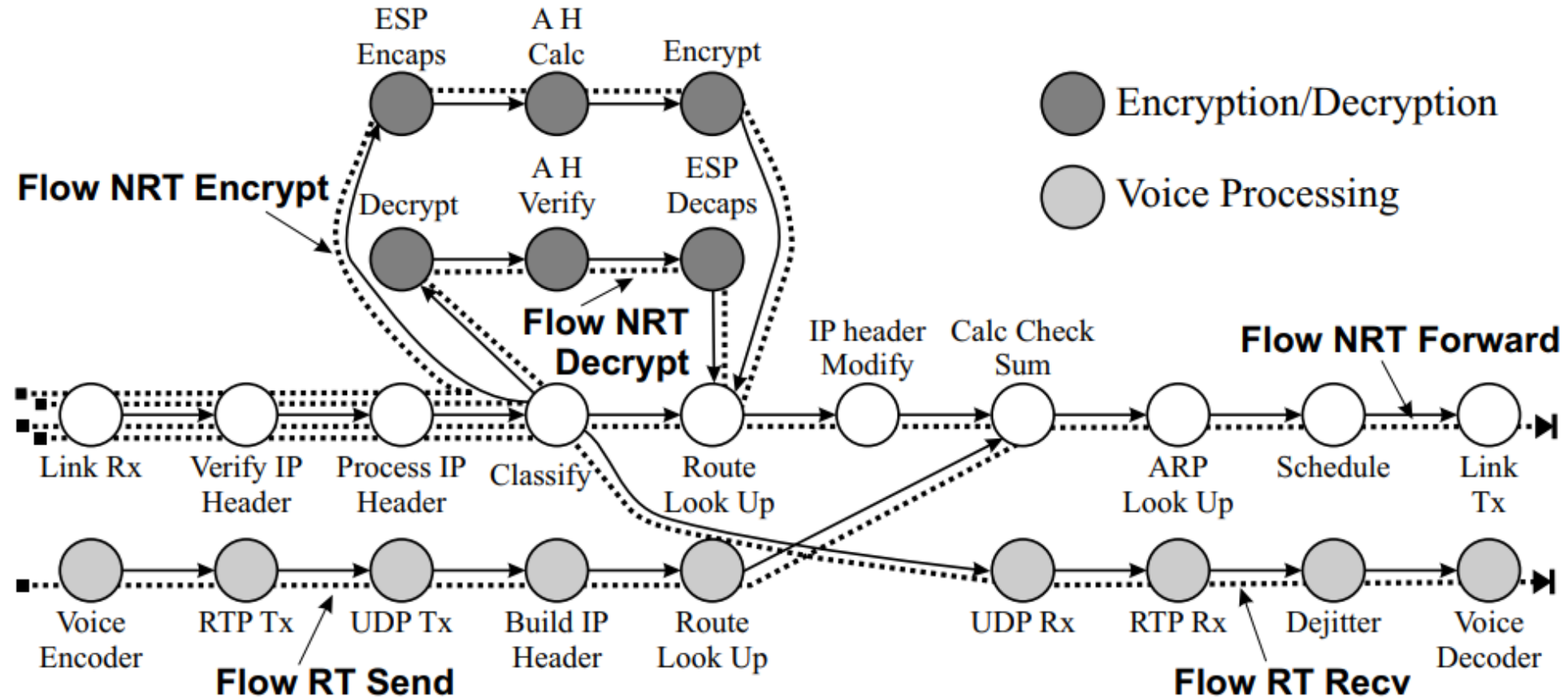
- **Possible use of linear optimization solver**
- **Let's have a look at this solutions:**
 - x_1 (area without fertilizer): 50.00 m²
 - x_2 (area with fertilizer): 50.00 m²
 - Harvest value (f_1): 425.00
 - Savings value (f_2): 325.00
- **This is a pareto-optimal solution...**
 - ... if you want to optimize the **sum** of f_1 and f_2
 - ... and **all** the land must be used for potatoes



Objective Space



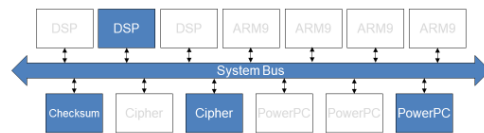
Example 2: Network Processor



Example 2: Network Processor

Case Study 1: Network Processor → Design Space

Allocation (selection of HW elements)



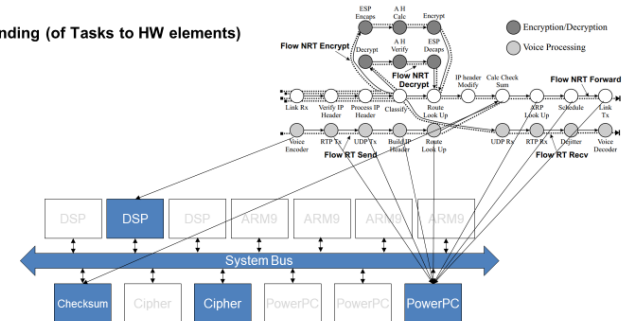
HS 2023

ESE – Dr. Simon Künzli, Tobias Welti

26

Case Study 1: Network Processor → Design Space

Binding (of Tasks to HW elements)



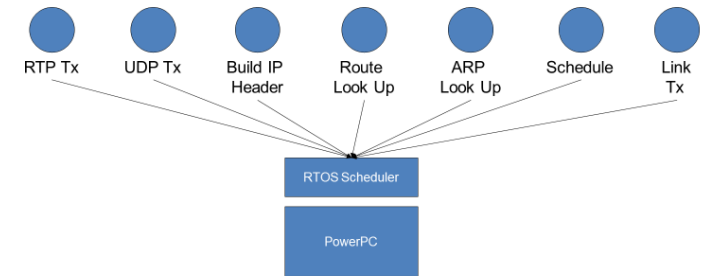
HS 2023

ESE – Dr. Simon Künzli, Tobias Welti

27

Case Study 1: Network Processor → Design Space

Runtime Environment (RTOS with Scheduling)



HS 2023

ESE – Dr. Simon Künzli, Tobias Welti

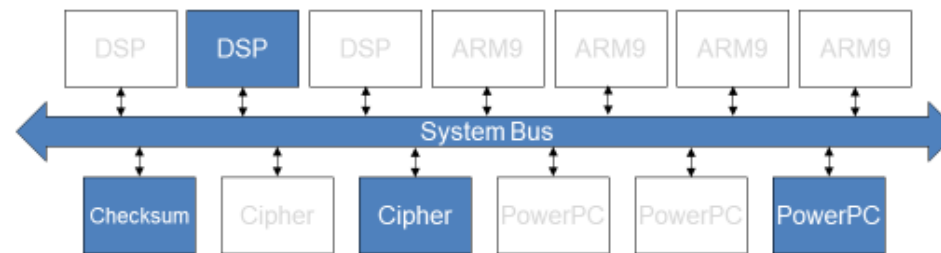
28

- Representing solutions is not always as easy as for a potato field with x_1 , x_2 being the areas treated without and with fertilizers...
- How would you represent a feasible solution in the decision space?

Representation of a Design – Allocation

Case Study 1: Network Processor → Design Space

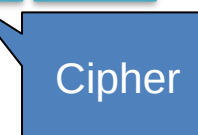
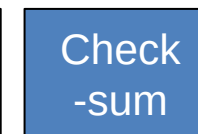
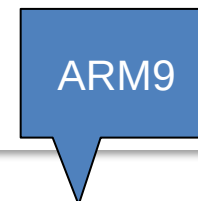
■ Allocation (selection of HW elements)



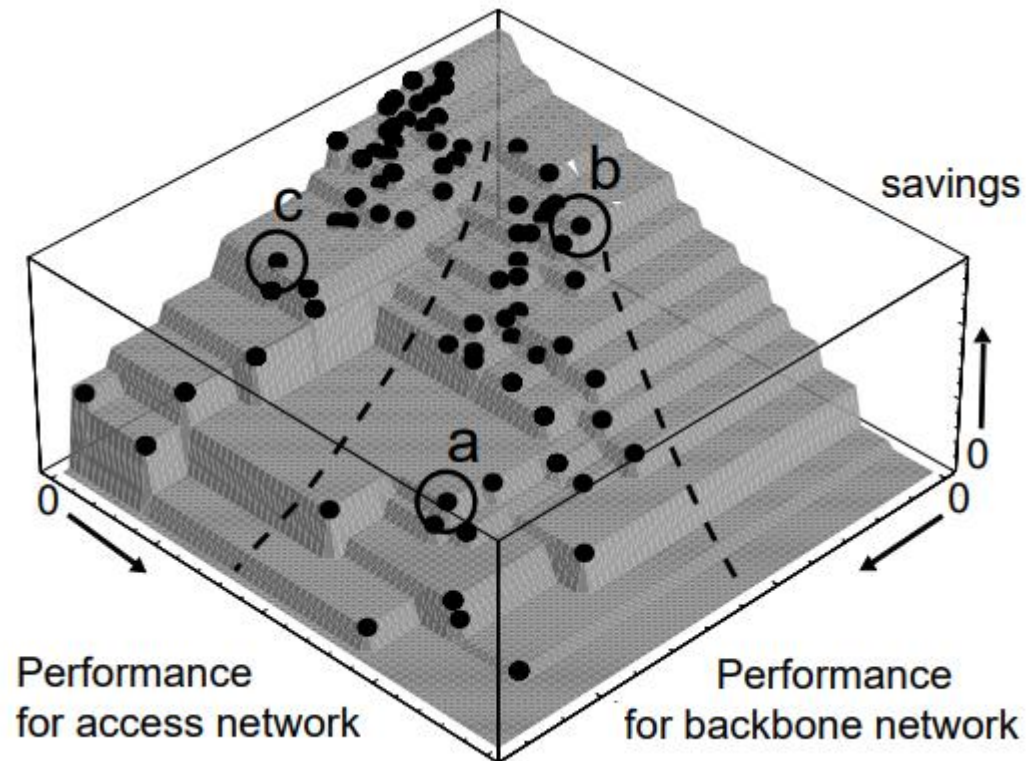
HS 2023

ESE – Dr. Simon Künzli, Tobias Welti

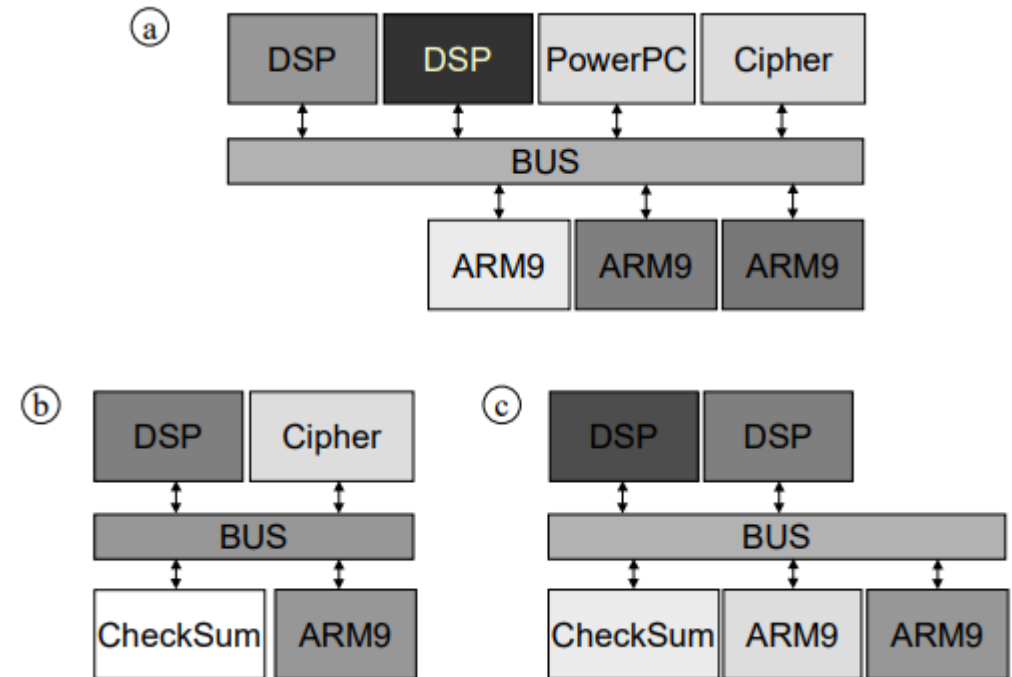
- For a solution x , $\rightarrow \text{Allocation}_x = (0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1)$



Example 2: Network Processor



Allocation_a = (1,1,0,1,1,1,0,0,0,1,0,0,1)



Allocation_b = (1,0,0,1,0,0,0,1,1,0,0,0,0)

Allocation_c = (1,1,0,0,0,1,1,1,0,0,0,0,0)

Example 2: Network Processor → Design Space

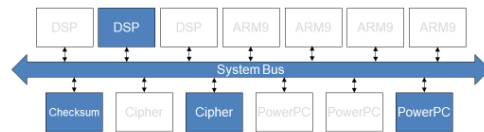


Design Representation – Allocation, Binding, RTE

Case Study 1: Network Processor → Design Space



Allocation (selection of HW elements)



HS 2023

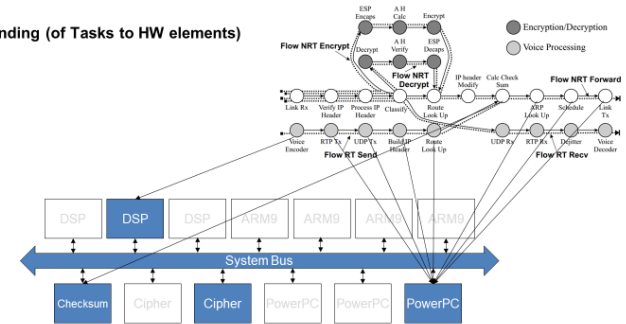
ESE – Dr. Simon Künzli, Tobias Welti

26

Case Study 1: Network Processor → Design Space



Binding (of Tasks to HW elements)



HS 2023

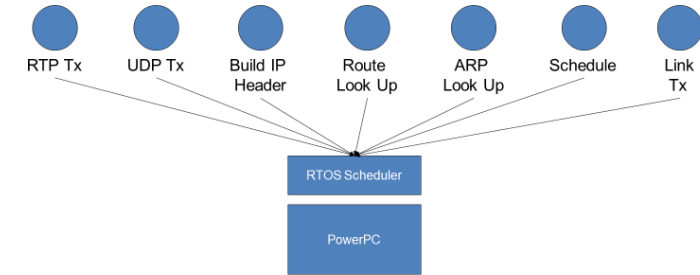
ESE – Dr. Simon Künzli, Tobias Welti

27

Case Study 1: Network Processor → Design Space



Runtime Environment (RTOS with Scheduling)



HS 2023

ESE – Dr. Simon Künzli, Tobias Welti

28

- A solution must contain the decisions taken for Allocation, Binding, and RTE
→ Add Binding_x and RTE_x to our solution description

I.e., we could use a binary representation using vectors for ALL the above steps:

- $\text{Solution}_1 = (0, 1, 0, 0, \dots, 1, 1, 1)$

- The advantage of such a representation is that it can be used with computers
- However, hard to read for humans ...

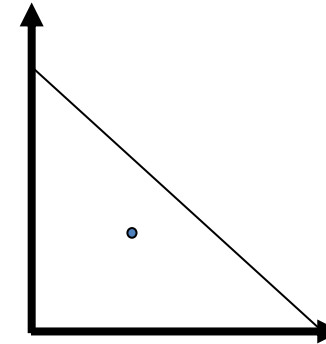
Searching for new solutions

- Find a new Vector $\text{Solution}_x = (\text{Allocation}_x, \text{Binding}_x, \text{RTE}_x)$

→ $\text{Solution}_x = (x_1, x_2, x_3, \dots, x_n)$, with $x_m \in \{0, 1\}$, for all $m=0..n$

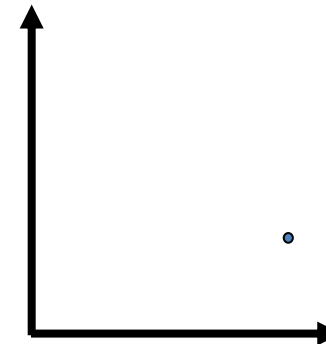
- Check whether Solution_x is feasible

- Is the solution in the decision space?
- Is it a valid solution?
- Does it meet all constraints?



- Evaluate feasible Solution_x for each of the m objectives, i.e. assign objective values $f_1..f_m$ for Solution_x

→ Evaluation: $f_y(\text{Solution}_x)$, for all $y=1..m$

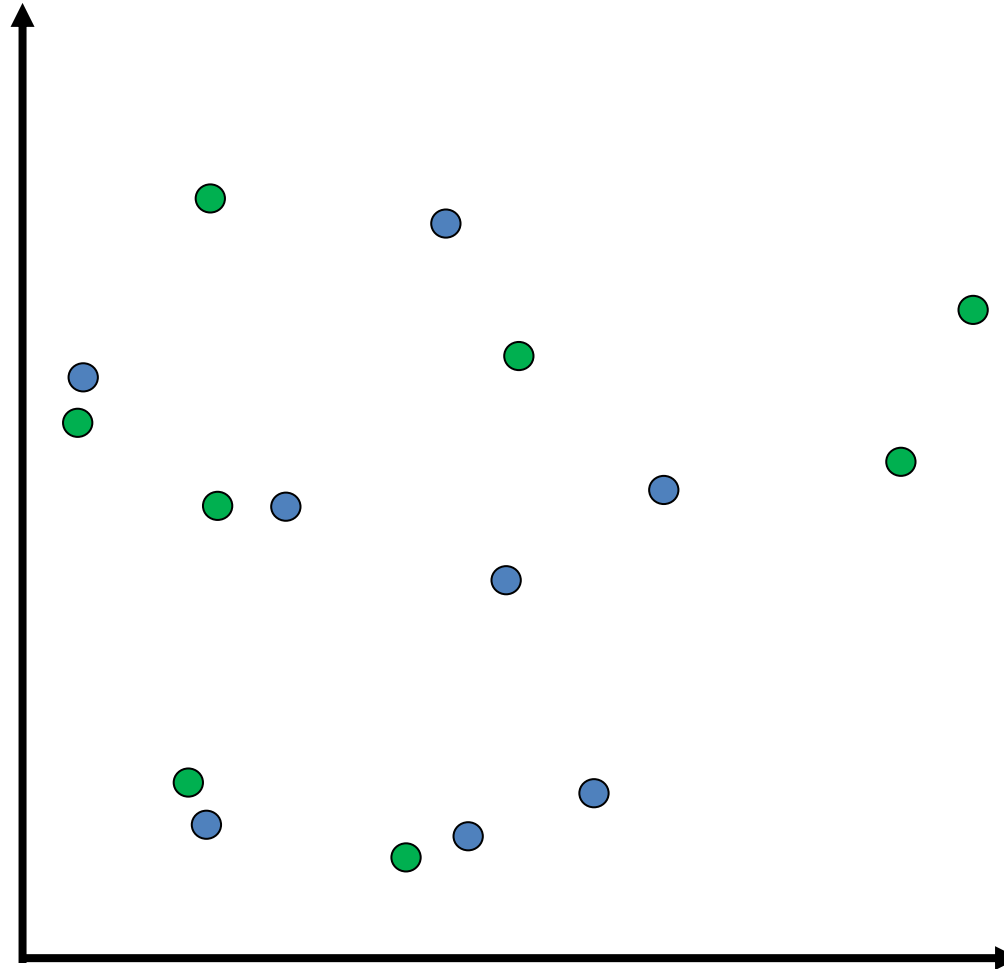


Search Design Space using Evolutionary Algorithms

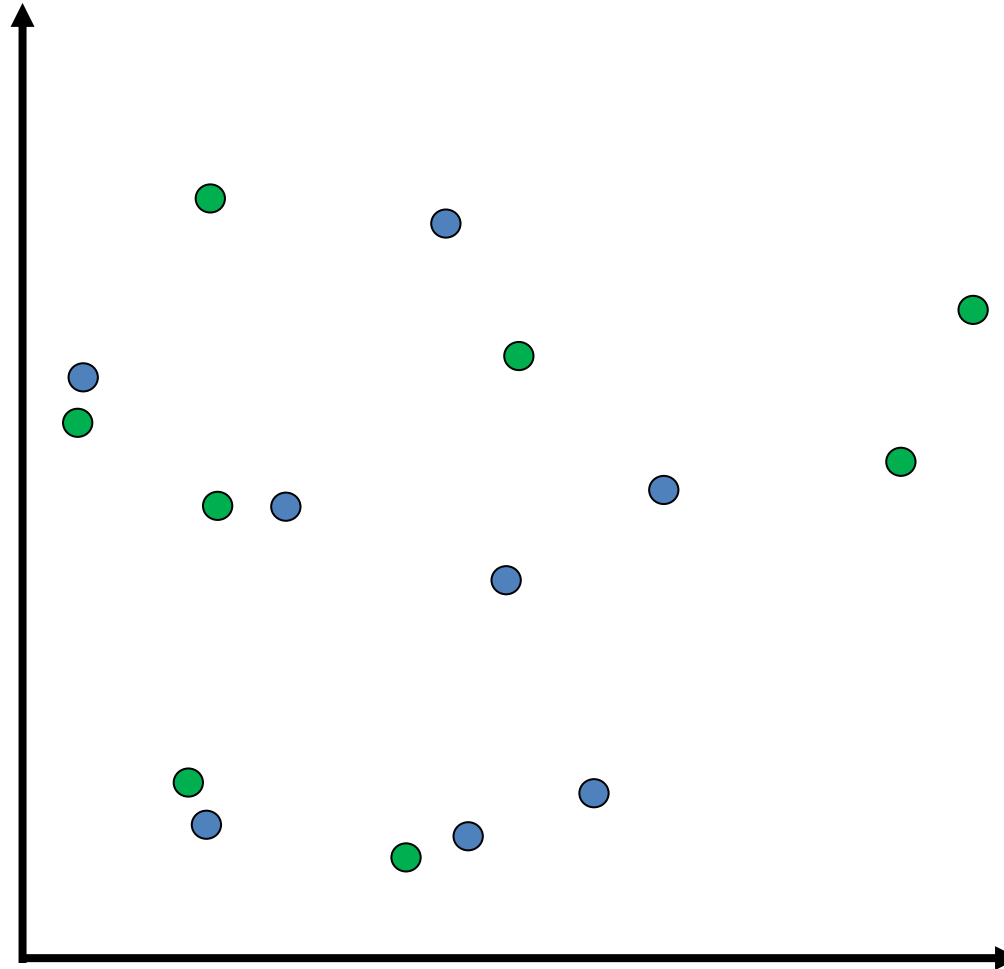
- **Well-suited search algorithm, if we have...**
 - ... a huge design space
 - ... limited resources
 - ... non-linear dependencies between decision space and the objective space

- **EAs keep a limited size *population* of «good» solutions, e.g., 100 solutions**

Search Design Space using Evolutionary Algorithms



Search Design Space using Evolutionary Algorithms

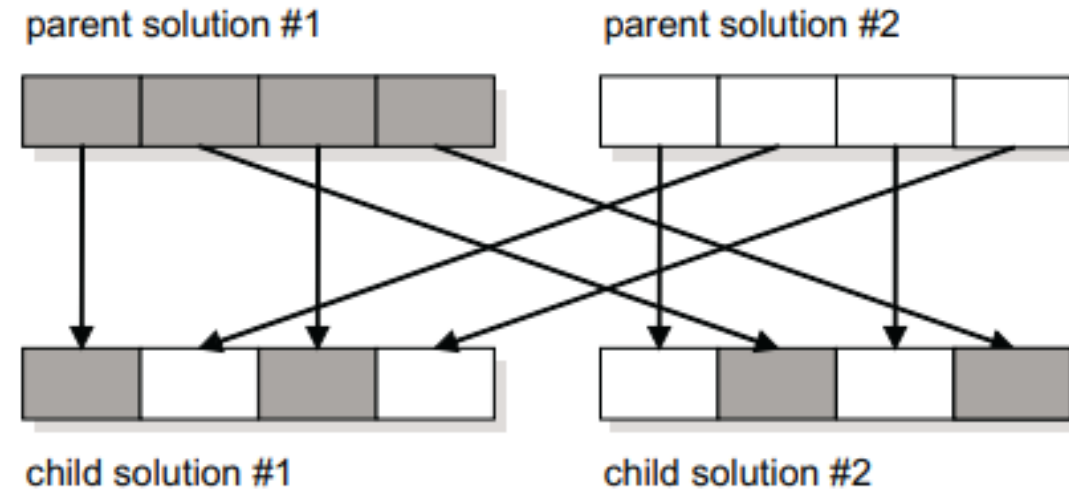
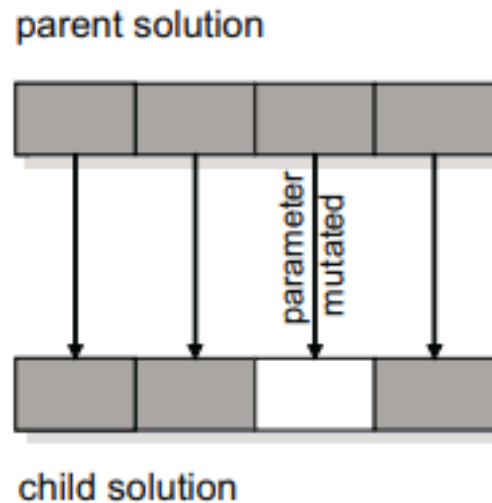


Search Design Space using Evolutionary Algorithms

- **Well-suited search algorithm, if we have...**
 - ... a huge design space
 - ... limited resources
 - ... non-linear dependencies between decision space and the objective space
- **EAs keep a limited size *population* of «good» solutions, e.g., 100 solutions**
- **Variation: From these solutions, the EA selects solutions for variation and derives a new generation (i.e. another 100 solutions)**
→ Total population now is 200 solutions
- **Selection: Reduce the population to 100 solutions again by removing *dominated* solutions from the population**

Variation

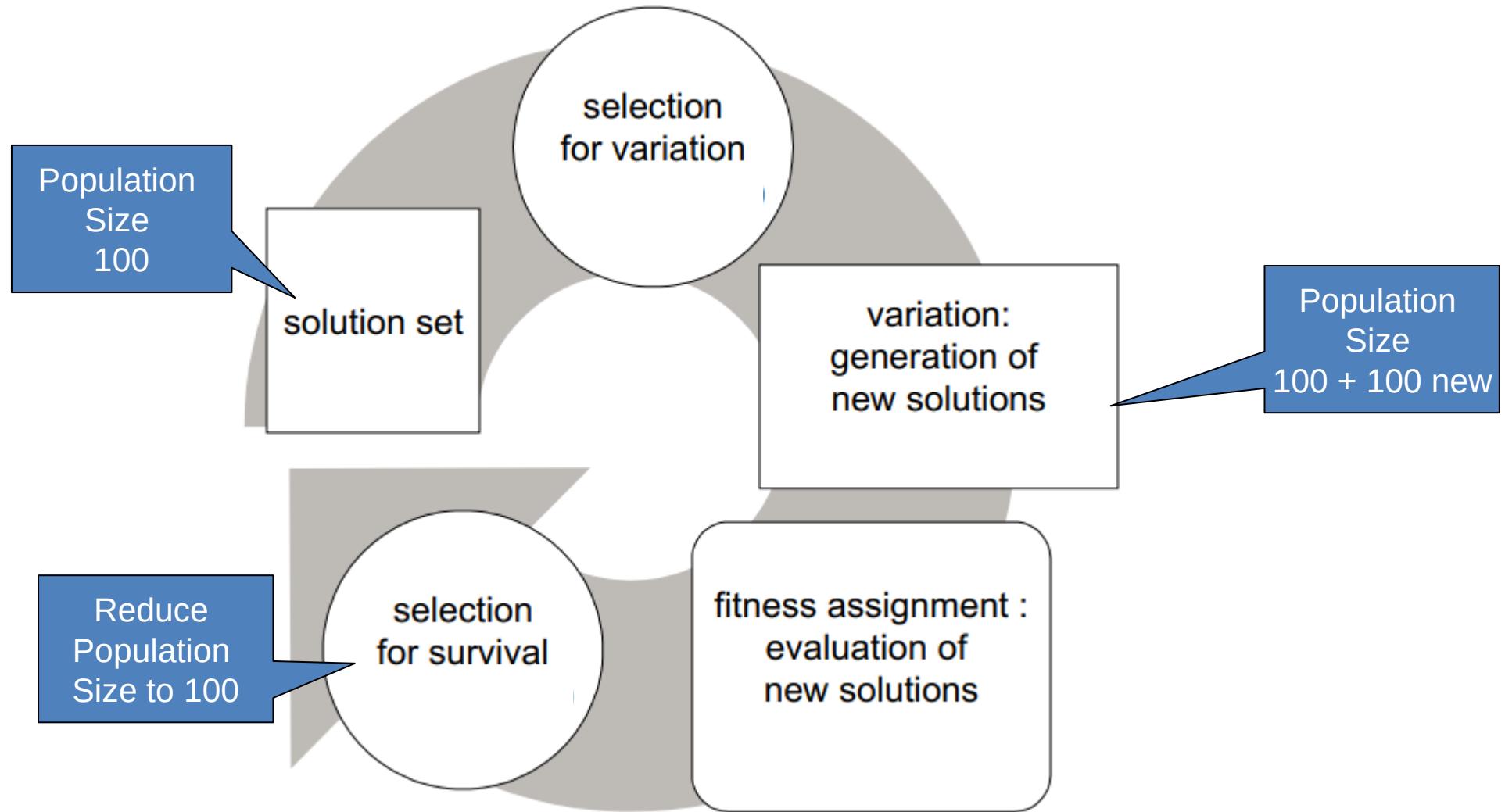
- In evolutionary algorithms, we use *mutation* and *recombination* of existing solutions to create offspring (→ the next generation solutions)



Selection

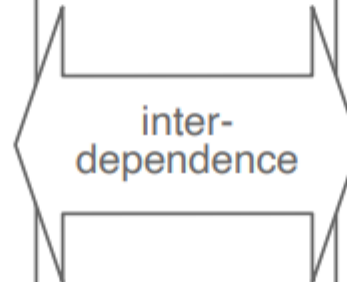
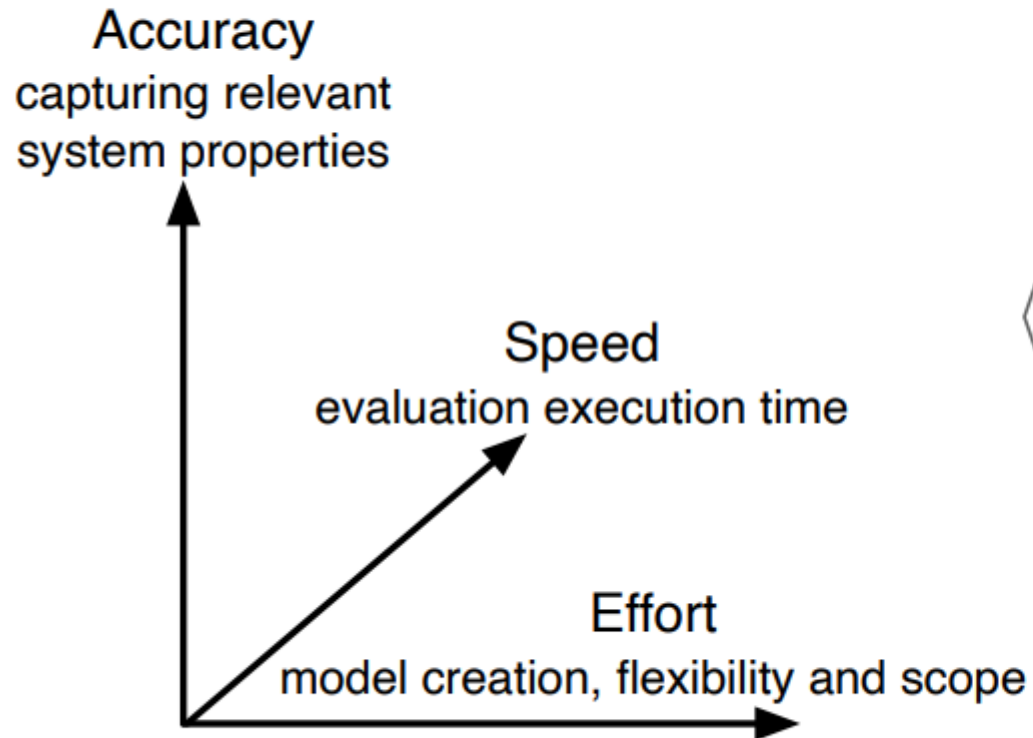
- **Evaluate all new solutions and derive objective values (→ fitness values)**
- **Derive for the whole population which solution set best dominates all other solution sets**
- **Select the best-dominating set as the new (reduced) population**
 - Actually, determining the best-dominating set is tricky, and making the difference between different evolutionary algorithms

“Evolutionary Loop”

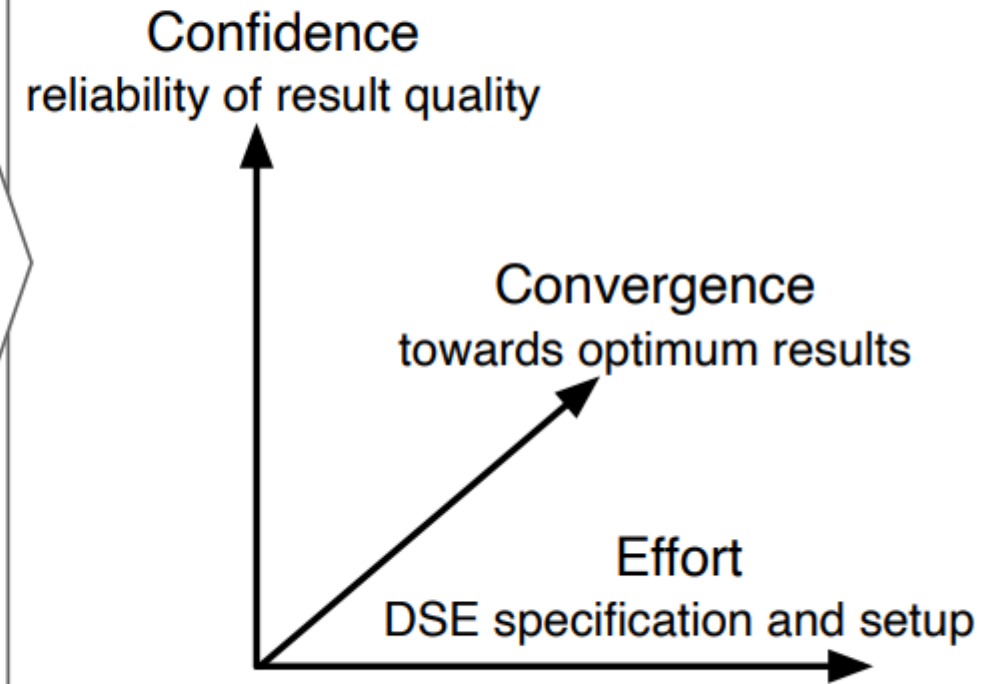


Summary: Design Space Exploration

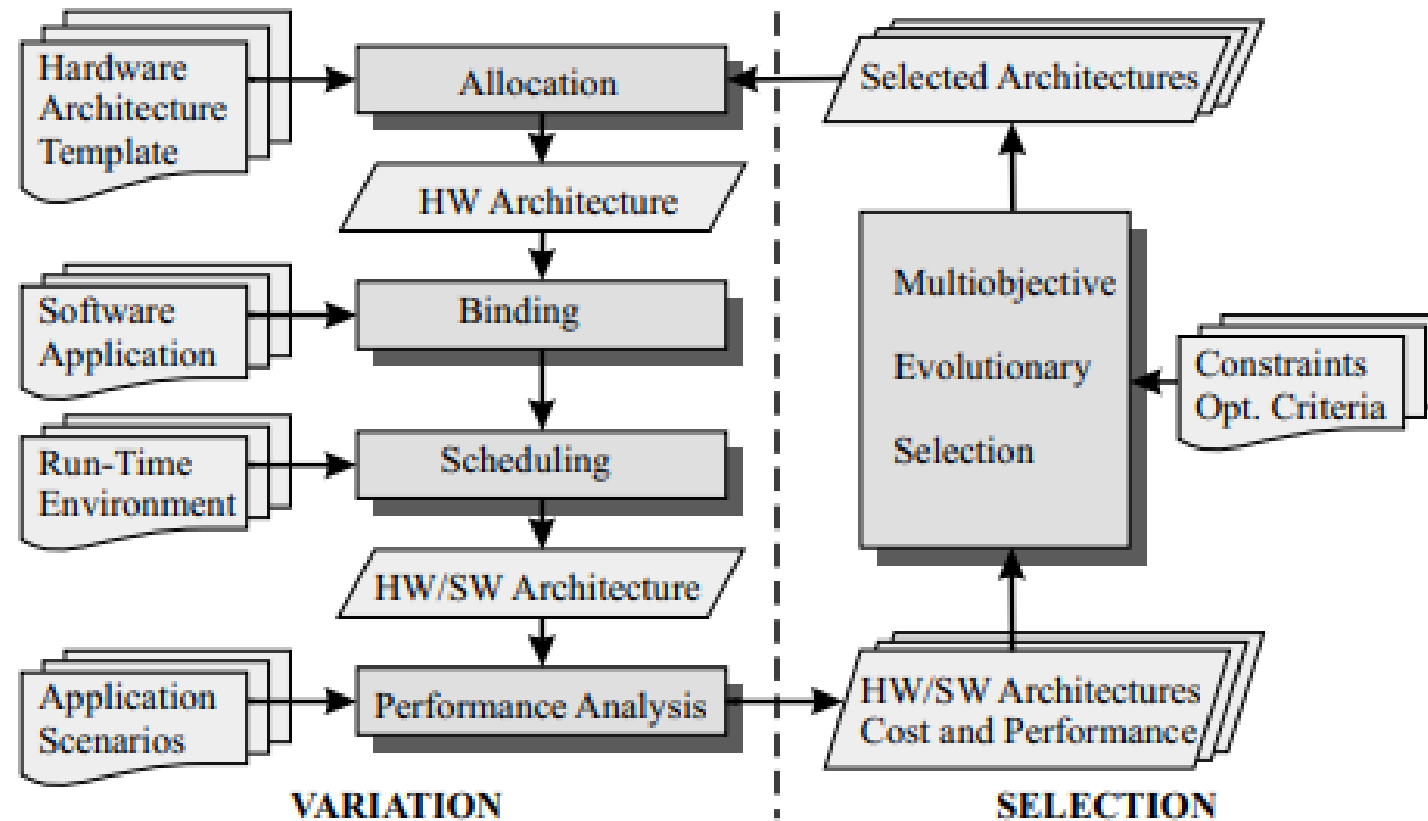
Evaluating a single design point



Searching the design space



Summary



Summary

- **Design Space Exploration can be performed at several abstraction layers**
- **Finding an optimal solution is not trivial, especially for multiobjective optimization problems**
- **The key elements to a successful DSE are:**
 - a suitable design representation (solution encoding)
 - a fast evaluation method (deriving objective values for a single solution)
 - a suitable search strategy (finding new possible solutions)
- **Evolutionary algorithms may help to conquer huge design spaces**